

Consigna para la gestión del repositorio del Proyecto de Fin de Asignatura (PFA)

1. Presentación

En el marco del curso Taller de Proyectos de Ingeniería de Sistemas e Informática, se establece el desarrollo de un proyecto orientado a la construcción de un Producto Mínimo Viable (PMV) de una aplicación web moderna para la generación de horarios universitarios.

El proyecto deberá desarrollarse en equipos de trabajo durante un periodo de 12 semanas, aplicando un enfoque sistémico para la identificación de necesidades, restricciones y actores involucrados, así como el uso del stack tecnológico.

2. Objetivo del Proyecto

Desarrollar un PMV funcional que permita resolver la problemática de la planificación de horarios universitarios, aplicando buenas prácticas de:

- a. Ingeniería de software
- b. Gestión ágil o híbrida de proyectos
- c. Control de versiones
- d. Organización arquitectónica del software
- e. Documentación técnica y de gestión

3. Lineamientos Técnicos y Metodológicos

3.1. Gestión del Repositorio (GitHub)

- a. Se deberá crear un repositorio en GitHub por equipo de trabajo.
- b. El repositorio deberá mantenerse activo, accesible y sin cambios de URL durante todo el periodo académico.
- c. El enlace del repositorio deberá ser compartido por el medio indicado por el docente.

3.2. Control de Versiones

- a. Se deberá evidenciar el uso de Git como sistema de control de versiones.
- b. Se deberá aplicar un flujo de trabajo, el cual deberá ser seleccionado y justificado por el equipo:
 - i. Git Flow
 - ii. Trunk-Based Development
 - iii. Feature Branch Workflow
- c. El repositorio deberá incluir:
 - i. Uso de ramas
 - ii. Commits significativos
 - iii. Integración de cambios mediante pull requests
- d. La ausencia de commits o ramas será considerada como incumplimiento metodológico.

3.3. Versionamiento del Software

- a. Se deberá aplicar el esquema de versionado semántico.
- b. Las versiones relevantes deberán etiquetarse mediante tags.



Taller de proyectos 2 – Ingeniería de Sistemas e Informática

- c. El PMV deberá estar identificado con la versión: v1.0.0

3.4. Organización del Código Fuente

- a. El proyecto deberá estructurarse obligatoriamente en:
 - Frontend (cliente)
 - Backend (servidor)
- b. La organización del código deberá basarse en:
 - Arquitectura por capas o
 - Arquitectura por características
- c. Consideraciones:
 - El Frontend representa la capa de presentación (UI/UX).
 - El Backend implementa la lógica de negocio y el acceso a datos mediante APIs.
- d. Se deberá evidenciar:
 - Separación de responsabilidades
 - Mantenibilidad
 - Escalabilidad

3.5. Buenas Prácticas de Desarrollo

- a. Uso obligatorio de archivo .gitignore.
- b. Exclusión de archivos innecesarios del repositorio.
- c. Código limpio, organizado y documentado.

4. Documentación del Proyecto

4.1. La documentación deberá elaborarse en formato Markdown y organizarse en la carpeta: docs/

4.2. La estructura deberá alinearse a las áreas de conocimiento de gestión del PMBOK:

- a. Inicio
- b. Planificación
- c. Ejecución
- d. Seguimiento y Control
- e. Cierre

4.3. Se podrán incluir carpetas adicionales según necesidad.

5. Contenido del Archivo README.md

El archivo README.md deberá contener obligatoriamente:

- 5.1. Nombre del proyecto
- 5.2. Tabla de contenido
- 5.3. Integrantes del equipo
- 5.4. Problemática abordada
- 5.5. Justificación del PMV
- 5.6. Tecnologías utilizadas

Taller de proyectos 2 – Ingeniería de Sistemas e Informática

5.7. Arquitectura del sistema

5.8. Instrucciones de instalación

5.9. Instrucciones de build

- a. Instrucciones de despliegue
- b. Enlace a video explicativo (máximo 5 minutos)
- c. Enlaces a la documentación ubicada en la carpeta docs/

6. Evidencia de Desarrollo

El repositorio deberá evidenciar:

6.1. Historial de commits

6.2. Evolución progresiva del PMV

6.3. Trabajo colaborativo entre los integrantes

7. Estructura Referencial del Repositorio

```
Repositorio_proyecto/  
├── docs/  
│   ├── inicio/  
│   ├── planificacion/  
│   ├── ejecucion/  
│   ├── seguimiento_control/  
│   ├── cierre/  
│   ├── otros/  
│   └── informe_final.pdf  
├── backend/  
│   └── tests/  
│   └── ...  
├── frontend/  
│   └── tests/  
│   └── ...  
├── otros/  
├── .env  
├── .gitignore  
└── README.md
```

8. Criterios de Evaluación y Penalización

Se considerarán los siguientes criterios:

- a. Documentación incompleta: será penalizada.
- b. Incumplimiento del versionado semántico: será penalizado.
- c. Falta de evidencia de trabajo colaborativo: será penalizada.
- d. Repositorios sin commits o ramas: serán considerados como incumplimiento metodológico.

9. Consideraciones Finales

9.1. La consigna deberá cumplirse en su totalidad.

9.2. El proyecto deberá reflejar la integración entre gestión, desarrollo y documentación.

9.3. El incumplimiento de los lineamientos afectará la evaluación del equipo.

Taller de proyectos 2 – Ingeniería de Sistemas e Informática

Rúbrica de evaluación - Repositorio y documentación del proyecto de Fin de Asignatura (PFA)

Criterio / Indicador	Sobresaliente (3)	Suficiente (2)	En desarrollo (1)	Insatisfactorio (0)
Gestión del repositorio en GitHub; creación, acceso continuo, URL estable y compartición oportuna del enlace	Repositorio correctamente configurado, accesible todo el periodo, URL estable, enlace entregado a tiempo y organizado con estándares profesionales	Repositorio creado y accesible, enlace compartido, sin problemas críticos	Repositorio con accesos intermitentes o enlace entregado fuera de plazo	No existe repositorio o no es accesible
Control de versiones; uso de ramas, commits significativos, pull requests y aplicación de flujo de trabajo definido	Uso consistente de ramas (>5), commits descriptivos (>20), PR correctamente integrados, flujo (Git Flow/Trunk/Feature) aplicado y documentado	Uso básico de ramas (≥2) y commits (>10), flujo aplicado parcialmente	Uso limitado de commits (<10) o sin estrategia clara de ramas	No hay evidencia de control de versiones o commits irrelevantes
Versionado semántico; uso de tags y etiquetado del PMV como v1.0.0	Versionado semántico correcto en múltiples versiones, uso adecuado de tags, PMV etiquetado correctamente como v1.0.0	Uso básico de versionado y al menos un tag correcto (v1.0.0)	Uso incorrecto o inconsistente de versionado, sin claridad en versiones	No aplica versionado ni uso de tags
Organización del código fuente; separación frontend/backend y arquitectura por capas o características	Estructura clara, frontend/backend bien definidos, arquitectura consistente, escalable y mantenible	Separación adecuada frontend/backend, arquitectura básica funcional	Estructura confusa o mezcla de responsabilidades	No existe organización clara del código
Buenas prácticas; uso de .gitignore, limpieza del repositorio y calidad del código	.gitignore completo, repositorio limpio, código legible, modular y documentado	.gitignore presente, código entendible con mínima documentación	.gitignore incompleto o código poco claro	No usa .gitignore o incluye archivos innecesarios
Documentación en carpeta docs; estructura basada en PMBOK (inicio, planificación, ejecución, seguimiento y cierre)	Documentación completa en todas las áreas PMBOK, bien estructurada, clara y detallada	Documentación presente en la mayoría de áreas, con contenido básico	Documentación incompleta o desorganizada	No existe documentación o es irrelevante
Contenido del README.md;	README completo con todos los	README incluye la	README incompleto o	No existe README o

Taller de proyectos 2 – Ingeniería de Sistemas e Informática

Criterio / Indicador	Sobresaliente (3)	Suficiente (2)	En desarrollo (1)	Insatisfactorio (0)
completitud, claridad, estructura y enlaces funcionales	apartados, bien redactado, enlaces funcionales y navegación clara	mayoría de secciones requeridas	con información poco clara	es irrelevante
Evidencia de desarrollo; historial de commits, evolución del PMV y trabajo colaborativo	Evidencia clara de evolución progresiva, contribución equilibrada del equipo, commits distribuidos	Evidencia de desarrollo con participación aceptable	Poca evidencia de colaboración o desarrollo irregular	No hay evidencia de trabajo colaborativo
Implementación del PMV; funcionalidad, coherencia con el problema y cumplimiento mínimo del producto	PMV funcional, resuelve el problema planteado, incluye mejoras adicionales o valor agregado	PMV funcional básico que cumple con el objetivo	PMV incompleto o con fallos funcionales	PMV no funcional o inexistente
Video explicativo; duración, claridad, demostración del sistema y acceso	Video ≤5 min, claro, bien estructurado, demuestra funcionalidades clave y accesible	Video cumple duración y muestra funcionalidad básica	Video poco claro, incompleto o excede duración	No presenta video o no es accesible